

The Product Manager's Guide to Mobile Apps

A practical, end-to-end reference on tracking, releasing, and the app lifecycle — with the iOS and Android nuances that actually trip teams up.

App Lifecycle

Analytics & Attribution

Release Management

iOS vs Android

App Size Optimization

Notifications

Play Store & App Store

Privacy & Compliance

Built as a shareable team reference — meant to be revisited section by section, not read in one sitting.
Scope: native consumer & B2B2C mobile apps. Concepts are platform-current; store policies and commercial terms evolve, so verify specifics against official docs before acting.

Contents

1. What Makes Mobile Different

Why the web playbook doesn't transfer cleanly

2. The App Lifecycle

Process states · cold/warm/hot start · background limits · what to instrument

3. Analytics & Tracking

Event taxonomy · attribution & deep links · the metric stack · tooling

4. Release Management

Versioning · beta tracks · staged rollout · feature flags · force-update · rollback reality

5. iOS vs Android: The Nuances

A side-by-side of the differences that change product decisions

6. App Size Optimization

Why size = installs · Android AAB/R8 · iOS App Thinning · tactics

7. Notification Handling

FCM & APNs · permissions · channels · deep links · best practices · metrics

8. Google Play & App Store

ASO · review · policy · ratings · consoles · billing

9. Privacy, Compliance & Permissions

Data safety, privacy labels, consent, regional law

10. PM Operating Checklists

Pre-launch · release day · post-launch · recurring health

11. Glossary

1 What Makes Mobile Different

If you came from web products, almost every instinct needs a small adjustment. The store sits between you and the user, releases are not instant or reversible, the OS can kill your app at will, and you are a guest on someone else's device subject to their permission and privacy rules.

The five structural differences

1. **You don't control distribution.** Apple and Google own the shelf. They review your build, can reject or remove it, set the commercial terms, and mediate updates. Your release timeline includes *their* timeline.
2. **Releases are slow and one-directional.** A web fix ships in minutes and reaches everyone. A mobile fix is a new binary, a new review, and a rollout to users who must *choose* to update. You cannot force users onto a new version, and you cannot truly "roll back" a version that is already on a phone.
3. **The OS controls your process.** Your app is suspended, throttled, or terminated based on memory, battery, and OS policy — not your wishes. State can vanish between sessions.
4. **You operate under a permission & privacy regime.** Location, camera, notifications, tracking identifiers — all gated by user consent and platform rules that have tightened sharply (ATT, runtime notification permission, privacy labels).
5. **Fragmentation is real, especially on Android.** Thousands of device/OS/screen/chipset combinations, wide range of network quality and storage, and a long tail of old OS versions. iOS is narrower but not uniform.

PM TAKEAWAY

Bake platform reality into your roadmap: assume a multi-day review buffer, design for users who never update, treat every permission as a conversion step you must earn, and never assume a feature behaves identically on both platforms until QA proves it.

2 The App Lifecycle

"Lifecycle" means two related things: the **runtime lifecycle** (the states your app moves through while the OS manages it) and the **product lifecycle** (build → release → adoption → deprecation). This section covers runtime; release lifecycle is Section 4.

2.1 App states & process lifecycle

Both platforms move an app through a small set of states. The names differ, the consequences are similar: when your app is not in the foreground, the OS can pause it and eventually reclaim its memory.

State	iOS	Android
Active / visible	Foreground Active — receiving events	Resumed (Activity) — in foreground, has focus
Visible but not focused	Foreground Inactive (e.g. during a call or split view)	Started / Paused — partially visible
Backgrounded	Background — brief execution, then Suspended	Stopped — not visible, retained if memory allows
Not running	Suspended → terminated by OS without notice	Destroyed — process may be killed to free memory

THE TRAP

A backgrounded app can be killed silently. When the user returns, they may get a **cold start** with lost in-memory state. If you didn't persist their progress (a half-filled form, a booking in flight), it's gone. State restoration is a product decision, not just an engineering one.

2.2 Cold, warm, and hot starts

- **Cold start:** process is not in memory; the OS creates it from scratch. Slowest. This is what new users and returning-after-kill users experience. **This is the start time that matters most for first impressions.**
- **Warm start:** process exists but the screen must be rebuilt. Faster.
- **Hot start:** app is already resident; just brought to foreground. Near-instant.

Slow cold start is a silent conversion killer. On Android, Play Console reports "excessive cold start" as a Vitals issue; on iOS, watch launch time in Xcode Organizer / MetricKit. Targets worth holding the line on: cold start under ~2s, and a visible first frame (not a blank/white screen) as fast as possible.

2.3 Background execution limits

Neither OS lets you run freely in the background — battery and privacy concerns dominate. This breaks naive assumptions like "I'll sync every minute" or "I'll track location continuously."

- **iOS** Background work is tightly budgeted: short background-fetch windows, background tasks the system schedules opportunistically, and special entitlements for audio, location, VoIP, etc. Don't assume periodic background execution.
- **Android** Doze mode, App Standby Buckets, and background-start restrictions throttle inactive apps. Long-running work must use **WorkManager** / foreground services with a visible notification. Background location is heavily restricted and requires extra review justification.

PM TAKEAWAY

Features that rely on "the app doing something while closed" (reminders, geofencing, sync, live tracking) are far harder and more constrained than they sound. Scope them with engineering early and have a server-driven fallback (push, scheduled jobs) rather than relying on the client staying alive.

2.4 What to instrument across the lifecycle

- App open (cold vs warm distinction)
- Time-to-interactive / first meaningful frame
- Crash & ANR (Android "App Not Responding")
- Foreground/background transitions & session length
- State-restoration success vs lost-state events
- Permission prompt shown / granted / denied
- Update prompt shown / accepted
- Network failures & offline usage

3 Analytics & Tracking

Mobile analytics is harder than web: there are no cookies, installs are mediated by the store, and platform privacy changes (ATT, ad-ID restrictions) have eroded the device-level identifiers attribution used to rely on. A PM needs a clean event model, a defensible attribution setup, and the right metric vocabulary.

3.1 Event taxonomy — design it before you ship

The single highest-leverage analytics decision is a consistent **event schema**. Renaming or restructuring events later means broken funnels and untrustworthy history. Decide naming conventions and a property dictionary up front and govern them.

- **Naming convention:** pick one style and enforce it — e.g. `object_action` (`appointment_booked` , `payment_completed`). Avoid free-for-all naming across teams.
- **Properties over event-explosion:** use `screen_viewed` with a `screen_name` property rather than 200 distinct screen events.
- **Identity:** define how you stitch anonymous → logged-in (a stable user ID), and how you handle multi-device.
- **Governance:** maintain a tracking plan (the source of truth), require it in PRDs, and review new events before launch. Treat the analytics schema like an API contract.

PM TAKEAWAY

Write the tracking plan as part of the PRD, not after. "We'll add analytics later" reliably produces gaps you discover only when an exec asks a question you can't answer.

3.2 Attribution & deep linking

Attribution answers "where did this install/event come from?" **Deep linking** answers "where inside the app should this user land?" They are intertwined because the link that drove the install also tells you the source.

The hard part: install attribution

On web, a click and a conversion share a cookie. On mobile, a click happens in a browser/ad, then the user goes to the store, then opens a freshly installed app — a broken chain. Two historical techniques bridged it:

- **Deterministic** — an exact identifier (e.g. device ad ID, or the Play Install Referrer) ties click to install. Most accurate, but increasingly restricted by privacy changes.
- **Probabilistic / fingerprinting** — statistical matching on IP + device signals within a time window. Less accurate and increasingly discouraged/restricted by platforms.

Platform privacy frameworks (this is what changed everything)

Aspect	iOS	Android
Tracking consent	ATT (App Tracking Transparency) — you must prompt for permission before using the IDFA to track across apps. Most users decline.	App tracking is allowed by default, but the Advertising ID can be reset/zeroed by the user; Google's Privacy Sandbox on Android is phasing toward privacy-preserving APIs.
Privacy-preserving attribution	SKAdNetwork / AdAttributionKit — aggregated, delayed, low-granularity conversion reporting without device-level IDs.	Play Install Referrer API for store installs; Privacy Sandbox Attribution Reporting API for aggregated measurement.
Practical impact	Per-user campaign attribution is largely gone; plan around aggregated, modeled data.	More signal than iOS today, but trending the same direction.

MMPs (Mobile Measurement Partners)

Tools like **AppsFlyer**, **Branch**, and **Adjust** sit between your app, the stores, and ad networks to normalize attribution and deep linking across the ecosystem. They handle SKAdNetwork/Install Referrer plumbing, deferred deep linking, and fraud filtering so you don't build it yourself.

Deep link types — know the difference

- **Standard deep link**: opens a specific screen *if the app is already installed*.
- **Deferred deep link**: user doesn't have the app → goes to store → installs → on first open, still lands on the intended screen (e.g. the specific product they tapped). Critical for campaign UX.
- **Universal Links** **iOS** / **App Links** **Android**: HTTPS links verified against your domain that open the app directly instead of the browser, with a graceful web fallback. The modern, recommended approach over custom URI schemes (`myapp://`).

3.3 The metric stack a PM should own

Organize metrics by the user journey (the "pirate metrics" AARRR framing works well):

Stage	Core metrics	What it tells you
Acquisition	Installs, install → open rate, cost per install (CPI), store listing conversion rate	Are you getting users, and efficiently?
Activation	Onboarding completion, time-to-first-value, "aha moment" rate	Do new users reach value quickly?
Retention	D1 / D7 / D30 retention, retention curves, cohort retention, resurrection rate	Do they come back? The truest health signal.
Engagement	DAU / WAU / MAU, stickiness (DAU/MAU) , sessions/user, session length, feature adoption	How deeply/often is the app used?
Revenue	ARPU, ARPPU, conversion to paid, LTV, churn, MRR (subscriptions)	Does usage turn into money?
Quality	Crash-free users/sessions, ANR rate, cold-start time, API latency/error rate	Is the experience stable? (Quality gates everything above.)

DEFINITIONS WORTH GETTING RIGHT

Stickiness = DAU/MAU (higher = more habitual use). **Retention curve** = % of a cohort still active by day N; a curve that flattens (rather than decaying to zero) signals product-market fit. **LTV** should be compared against blended CAC; a viable business needs LTV > CAC with a payback period you can fund.

3.4 The tooling landscape

Category	Common tools	Use for
Product analytics	Mixpanel, Amplitude, GA4 / Firebase Analytics	Funnels, retention, cohorts, behavioral events
Attribution (MMP)	AppsFlyer, Branch, Adjust	Install attribution, deep links, ad-network measurement
Crash & performance	Firebase Crashlytics, Sentry, Instabug	Stability, crash/ANR triage, perf traces
A/B & remote config	Firebase Remote Config, Optimizely, LaunchDarkly, Statsig	Experiments, feature flags, rollout control
Session replay / qual	UXCam, Smartlook, FullStory	Watching real sessions, rage-tap/funnel-drop diagnosis

3.5 Privacy & consent (you can't track without it)

- **ATT** **ios** gates cross-app tracking; design a clear pre-permission "priming" screen explaining the value before the system prompt.
- **Consent management** (GDPR in EU, India's DPDP Act, CCPA in California) governs what you may collect and share. Many analytics SDKs support a consent-mode toggle — wire it up.
- **Data minimization**: collect what you'll actually use. It reduces compliance surface, store-review friction, and the privacy-label disclosures you must make.

4 Release Management

Shipping a mobile app is a controlled, partially irreversible process gated by store review. The craft is in releasing safely — small blast radius, the ability to dial a change up or down without a new build, and a plan for when something breaks on a version already in users' hands.

4.1 Versioning & build numbers

Concept	iOS	Android
User-visible version	Version string (e.g. 3.4.0 , semantic versioning)	versionName (e.g. 3.4.0)
Internal incrementing build	Build number (CFBundleVersion)	versionCode (integer; Play requires it to always increase)
Key rule	Each upload needs a higher build number	Each release needs a strictly higher versionCode — you cannot reuse or go backwards

Semantic versioning (MAJOR.MINOR.PATCH) communicates intent: breaking change, new feature, or fix. Useful for users, support, and your own changelog discipline.

4.2 Environments & CI/CD

A healthy pipeline separates **dev** → **staging/QA** → **production** builds, automates build/sign/test, and distributes betas automatically. Common tooling: Fastlane (automation), Bitrise / Codemagic / GitHub Actions (CI), plus automated signing. The PM value: faster, more reliable releases and less "it works on my machine" risk.

4.3 Beta & pre-release distribution

Channel	iOS	Android
Internal team builds	TestFlight (internal testers, up to the platform limit; fast)	Play Console Internal testing track
Wider beta	TestFlight external testers (requires a lightweight Beta App Review)	Closed & Open testing tracks
Cross-platform alt	Firebase App Distribution — distribute builds to testers on both platforms outside the stores	

4.4 Staged / phased rollouts

- **Android** **Staged rollout**: release to a % of users (e.g. 1% → 5% → 20% → 100%), monitor Vitals, and **halt** if crash/ANR spikes. You can pause and resume.
- **iOS** **Phased release**: automatic 7-day ramp to all users for app-update; you can pause, but the percentages are system-defined and less granular than Android.

REALITY CHECK

A staged rollout limits how many users get a bad build — but the users who already updated **cannot be downgraded**. Staged rollout reduces blast radius; it is not a rollback.

4.5 Feature flags & remote config

The single most important release-safety tool. Ship code **dark** (disabled), then enable it server-side — no new build, no review.

- **Decouple release from deploy:** the binary can be live for weeks before a feature is switched on.
- **Kill switch:** if a feature misbehaves, turn it off instantly for everyone, even on old binaries.
- **Gradual exposure & targeting:** enable for 5%, internal users, a region, or a cohort.
- **Cleanup discipline:** stale flags become tech debt — track and remove them.

4.6 A/B testing & experimentation

Run controlled experiments on flagged variants. Mobile caveats: (1) you can only experiment on code that already shipped in the installed binary; (2) sample sizes accrue slower than web because adoption of a new version takes time; (3) define success metrics and guardrail metrics (e.g. don't win conversion while tanking crash-free rate) before you start.

4.7 Force update, soft update & kill switch

Because users may never update, you need a server-controlled version policy:

- **Soft update:** a dismissible "new version available" nudge.
- **Force update:** a blocking screen for versions below a minimum supported version — reserved for security fixes, breaking API changes, or critical bugs. Use sparingly; it's a hard interruption. (Android offers in-app updates; iOS typically requires your own version-gate logic calling the store.)
- **Kill switch:** remotely disable a specific broken feature or endpoint without forcing an app update.

4.8 Hotfixes & the rollback myth

INTERNALIZE THIS

You cannot un-ship a mobile version. "Rollback" on mobile means one of: (a) halt the rollout so *fewer* new users get the bad build; (b) flip a feature flag / kill switch off; (c) push a server-side fix; or (d) submit a **new higher version** with the fix (subject to review, possibly expedited). There is no instant global revert like the web. This is why flags and staged rollouts are non-negotiable.

5 iOS vs Android: The Nuances

A consolidated side-by-side of the differences that actually change product, design, and planning decisions. Keep this handy when scoping any cross-platform feature.

Dimension	iOS	Android
Store & review	App Store; human review, can be slower/stricter; rejections common over guidelines	Google Play; more automated review, historically faster; policy enforcement has tightened
Distribution model	App Store (+ EU alternative marketplaces due to DMA); sideloading limited	Play + sideloading + alternative stores openly supported
Build artifact	IPA via App Store Connect	AAB (Android App Bundle) required for Play; Play generates per-device APKs
Versioning	Version + build number	versionName + ever-increasing versionCode
Rollout control	Phased release (7-day, system-paced)	Staged rollout (you set %), pause/resume
Update enforcement	No native forced update; build your own gate	In-app updates API (flexible & immediate)
Notifications	APNs; opt-in permission always required; provisional auth option; interruption levels & Focus	FCM; runtime permission only since Android 13; channels (since 8.0) users control granularly
Background work	Very restrictive; system-scheduled windows	Restrictive (Doze, buckets) but foreground services available
Tracking ID	IDFA gated by ATT (mostly off)	Advertising ID (resettable); Privacy Sandbox emerging
Size optimization	App Thinning (slicing, on-demand resources)	AAB dynamic delivery, R8 shrinking, Play Asset Delivery
Fragmentation	Narrow device range; fast OS adoption	Huge device/OS/screen spread; slow OS adoption tail
Design language	Human Interface Guidelines; tab bars, swipe-back	Material Design; back button/gesture, FABs
Payments	StoreKit IAP; commission & external-link rules evolving under regulation	Play Billing; user-choice/external billing options expanding under regulation
Update adoption	Faster; users update readily	Slower; long tail on old versions

PM TAKEAWAY

"Build it once for both" is a myth at the edges. Notifications, deep links, background behavior, payments, and permissions all diverge. Scope each platform's variant explicitly and never assume parity until tested on real devices.

6 App Size Optimization

App size is a growth metric, not just an engineering nicety. Larger apps convert fewer installs — users on limited storage, slow networks, or metered data abandon big downloads. This matters acutely in markets like India and other emerging regions.

6.1 Why size moves the needle

- **Install conversion:** every additional MB measurably reduces install-completion rate; the effect compounds on slow/expensive networks.
- **Cellular thresholds:** historically the App Store warned/blocked very large downloads over cellular (the threshold has shifted over time), nudging users to "wait for Wi-Fi" — which often means never.
- **Storage pressure & uninstalls:** when a phone runs low on space, big apps are first to be deleted.
- **Update fatigue:** large frequent updates discourage users from staying current (which then hurts your release strategy).

6.2 Android tactics

- **Android App Bundle (AAB):** required by Play; Google generates optimized APKs per device (screen density, CPU architecture, language) so users download only what their device needs — the single biggest lever.
- **R8 / ProGuard:** code shrinking, obfuscation, and dead-code removal. Enable resource shrinking to strip unused resources.
- **Play Feature Delivery:** split rarely used features into **dynamic feature modules** delivered on-demand, conditionally, or at install time.
- **Play Asset Delivery:** for games/large assets, serve large packs through Play rather than bundling them.
- **Image/asset hygiene:** use **WebP** (or AVIF) over PNG/JPEG; vector drawables for icons; remove unused densities and locales.
- **Audit:** Android Studio's **APK Analyzer** shows exactly what's bloating the binary.

6.3 iOS tactics

- **App Thinning:** the umbrella mechanism — **slicing** delivers only the assets/code for the user's specific device variant.
- **On-Demand Resources (ODR):** host tagged resources that download when first needed and can be purged later, keeping the initial download small.
- **Asset catalogs & image optimization:** let Xcode optimize and slice image assets per device.
- **Audit:** review the App Store-reported install size and the App Thinning size report in Xcode Organizer. (Bitcode is deprecated and no longer the lever it once was.)

6.4 Cross-cutting tactics

- Compress & right-size all images/media
- Lazy-load heavy content from the server
- Audit third-party SDKs — each adds size & risk
- Remove unused languages/locales/assets
- Prefer vector over raster where possible
- Strip debug symbols from release builds
- Deduplicate libraries pulled in twice
- Track size as a release metric over time

PM TAKEAWAY

Put app size on your release dashboard and set a budget. Treat a size regression in a release the same way you'd treat a crash regression — ask what added the weight and whether it's worth the install-conversion cost.

7 Notification Handling

Notifications are your most powerful re-engagement channel and your fastest way to get uninstalled. The discipline is earning the opt-in, then sending notifications that are timely, relevant, and rare enough to stay welcome.

7.1 The delivery architecture

Layer	iOS	Android
OS push service	APNs (Apple Push Notification service)	FCM (Firebase Cloud Messaging)
Typical setup	Your server → FCM (which can route to both Android and, via APNs, to iOS) → device. FCM is the common cross-platform sending layer.	

7.2 Notification types

- **Push (remote):** server-initiated, arrives even when the app is closed. The main re-engagement tool.
- **Local:** scheduled by the app on the device (reminders, alarms) — no server needed.
- **Silent / data (background) push:** wakes the app to sync data without showing a banner. Throttled by the OS; not guaranteed delivery.
- **Rich notifications:** images, action buttons, expandable content.
- **In-app messages:** shown *inside* the app while it's open — not OS notifications, no permission needed, great for contextual nudges.

7.3 Permissions — the opt-in is everything

Aspect	iOS	Android
Permission required?	Always — explicit opt-in before any push	Runtime permission (POST_NOTIFICATIONS) required since Android 13 ; granted by default below that
Soft-ask option	Provisional authorization — deliver quietly to Notification Center without an upfront prompt, then earn full opt-in	No provisional tier; prime with your own pre-permission screen
User-side control	On/off + per-type via interruption levels; Focus modes	Notification channels (since 8.0) — users mute/allow categories individually

OPT-IN PRIMING — THE HIGHEST-LEVERAGE MOVE

Never fire the system permission prompt cold on first launch. Show a **pre-permission screen** first that explains the value ("Get notified when your appointment is confirmed"), and only trigger the real OS prompt for users who say yes. A declined system prompt is sticky — on iOS you generally don't get a second chance in-app, the user must go to Settings.

7.4 Android notification channels

Since Android 8.0, every notification must belong to a **channel** (e.g. "Order updates", "Promotions"). Users control each channel independently — they can mute your promos while keeping transactional alerts. **Design your channels deliberately:** separate transactional from marketing so users don't nuke everything at once.

7.5 Deep linking from notifications

A notification should land the user exactly where the content lives, not on the home screen. Wire each notification's payload to a deep link (Section 3.2). A tap that dumps the user at a generic landing screen wastes the engagement and trains them to ignore you.

7.6 Best practices

- **Relevance & segmentation:** target by behavior and lifecycle stage, not blasts to everyone.
- **Timing:** respect time zones and quiet hours; send when the action is useful.
- **Frequency caps:** cap sends per user per day/week; over-notifying is the top uninstall driver.
- **Separate transactional from promotional** (and let users keep one while muting the other).
- **Personalize** content and send-time where you can.
- **Always deep link** to the relevant destination.

7.7 Metrics to watch

Metric	What it tells you
Opt-in rate	Quality of your permission priming & trust
Delivery rate	Token health, OS throttling, deliverability
Open / tap rate	Relevance & copy quality
Conversion from notification	Did the tap drive the intended action?
Opt-out / channel-mute / uninstall after send	Are you over-sending or being irrelevant?

8 Google Play & App Store

The store is your storefront, your distribution channel, your billing system, and your gatekeeper. A PM should understand the listing, the review process, the policy surface, and the console capabilities on both platforms.

8.1 Store presence & ASO

App Store Optimization is the mobile equivalent of SEO — improving discoverability and listing conversion.

- **Discoverability levers:** title, subtitle/short description, keywords (iOS has a dedicated keyword field; Play indexes the description), category.
- **Conversion levers:** icon, screenshots, preview video, ratings & reviews, and the first lines users see before "more".
- **Test it:** Play **store listing experiments** and Apple **Product Page Optimization** let you A/B test listing assets.

8.2 Listing asset requirements (differ by platform)

Each store mandates specific icon sizes, screenshot dimensions per device class, feature graphics (Play), preview videos, and copy fields with character limits. **Maintain a localized asset kit** and check current spec sheets before each submission — requirements change.

8.3 Review process & common rejections

- **iOS** Human review; stricter and can reject for guideline nuances. Build in a review buffer; request **expedited review** only for genuine emergencies.
- **Android** More automated; historically faster, though new-developer and sensitive-permission apps face more scrutiny.

Frequent rejection causes: broken/incomplete functionality, privacy/permission issues, misleading metadata, payment-policy violations (using non-store payments for digital goods), insufficient content for the category, crashes on review, and missing demo credentials for gated features.

PM TAKEAWAY

Give reviewers everything: working test accounts, clear review notes explaining gated/region-locked features, and a build that doesn't crash. Most rejections are avoidable friction, and each one costs you a review cycle.

8.4 Policy & data disclosure

Requirement	iOS	Android
Privacy disclosure	Privacy Nutrition Labels & app privacy details on the listing	Data safety section in Play Console
Must declare	What data you collect, why, whether it's shared, and how it's protected — must match actual behavior or risk removal	
Sensitive permissions	Justify usage strings (e.g. location, camera) clearly	Declaration forms for sensitive permissions (e.g. background location, SMS)

8.5 Ratings & reviews

- Use the **in-app review APIs** (SKStoreReviewController on iOS, In-App Review on Android) to prompt at a happy moment — not after a crash.
- Both stores let you **respond** to reviews; treat it as a support & signal channel.
- Ratings feed ranking and conversion — a half-star swing moves installs.

8.6 Console capabilities a PM uses

Capability	Play Console	App Store Connect
Health monitoring	Android Vitals (crashes, ANRs, cold start, battery, excessive wakeups)	Crash reports, MetricKit metrics, Organizer
Acquisition data	Store listing acquisition, conversion funnels	App Analytics (impressions, conversion, retention)
Release tooling	Testing tracks, staged rollout, in-app updates	TestFlight, phased release
Experiments	Store listing experiments	Product Page Optimization & custom product pages

8.7 Monetization & billing (PM lens)

- **Digital goods generally must use store billing** (StoreKit / Play Billing), which historically carried a commission (~30%, with reduced tiers for small developers and subscriptions after year one). **These commercial terms and external-payment rules are actively changing under regulation (EU DMA, US litigation, and others)** — verify current rules before modeling revenue.
- **Physical goods & services** (e.g. booking a real-world appointment, ordering food) typically use normal payment processors, not store billing.
- **Subscriptions:** both stores have rich subscription tooling (intro offers, free trials, grace periods, billing-retry). Track trial-to-paid conversion and involuntary churn (failed payments).

9 Privacy, Compliance & Permissions

Privacy is now a first-class product constraint. Permissions are conversion steps, disclosures are mandatory and audited, and regional laws dictate what you may collect. Getting this wrong risks store removal and legal exposure.

9.1 The permission model

- Sensitive capabilities (location, camera, mic, contacts, notifications) require **runtime user consent**, and increasingly granular options (e.g. precise vs approximate location; photo-library "selected photos only").
- **Request in context, not upfront:** ask for camera access when the user taps "scan", not on launch. Pair with a priming explanation.
- **A denied permission is sticky.** Plan a graceful degraded experience and a path to re-enable via Settings.

9.2 Mandatory disclosures

Both stores require you to declare your data practices (Apple Privacy Labels, Play Data safety). These must **match real behavior** — including what your third-party SDKs do. An analytics or ad SDK collecting more than you disclosed is your liability.

9.3 Regional laws to know

Law	Region	PM-relevant gist
GDPR	EU/EEA	Lawful basis & consent for personal data; access/deletion rights; strict on tracking
DPDP Act	India	Consent-based processing, notice, data-principal rights, purpose limitation
CCPA/CPRA	California	Disclosure, opt-out of "sale/share", deletion rights
HIPAA	US (health)	Safeguards for protected health information — critical for health apps & their vendors

PM TAKEAWAY

Treat every SDK you add as a privacy & compliance decision, not just an engineering one. Keep a living inventory of what each SDK collects, ensure it's reflected in your store disclosures, and minimize by default. In regulated domains (health, finance), loop in legal/compliance before the build, not after.

10 PM Operating Checklists

Reusable checklists to run against any release. Adapt to your team's process; the point is to make the non-obvious mobile pitfalls explicit.

10.1 Pre-launch (feature/release)

- ☐ PRD includes the tracking plan (events, properties, success metrics)
- ☐ Feature is behind a flag / remote config with a kill switch
- ☐ Deep links defined and tested (incl. deferred deep links for campaigns)
- ☐ Permissions requested in-context with priming; denied-state handled
- ☐ Notification channels/types defined; opt-in priming in place
- ☐ State restoration handled (no lost progress on cold start)
- ☐ Tested on real low-end Android devices & older OS versions
- ☐ App size impact measured against budget
- ☐ Store listing assets & metadata updated & localized
- ☐ Privacy labels / Data safety updated to match behavior
- ☐ Crash & analytics SDKs verified firing in the release build
- ☐ Reviewer notes + working test accounts prepared

10.2 Release day

- ☐ Staged/phased rollout started at a small % (Android: 1–5%)
- ☐ Dashboards open: crash-free rate, ANR, cold start, key funnel
- ☐ Feature flag default state confirmed
- ☐ Rollout-halt and kill-switch criteria agreed in advance
- ☐ On-call / owner identified for the rollout window

10.3 Post-launch (first 24–72h)

- ☐ Crash-free users/sessions holding vs baseline
- ☐ ANR & cold-start within targets (Android Vitals / iOS metrics)
- ☐ Adoption curve of the new version tracking as expected
- ☐ Target funnel + guardrail metrics moving the right way
- ☐ Store reviews & ratings monitored for new-version complaints
- ☐ Rollout advanced to next % only after metrics clear

10.4 Recurring health review

- ☐ Retention curves & stickiness (DAU/MAU) by cohort
- ☐ Crash-free rate & ANR trend over time
- ☐ App size trend across releases
- ☐ Notification opt-in, open, and opt-out/uninstall trends
- ☐ Version fragmentation (% on min-supported & below)

- ☐ Stale feature flags identified for cleanup
- ☐ Store rating trend & review themes

11 Glossary

Term	Meaning
AAB	Android App Bundle — publishing format Play uses to generate optimized per-device APKs
ANR	Application Not Responding — Android event when the UI thread blocks too long
APNs	Apple Push Notification service
ASO	App Store Optimization — improving store discoverability & conversion
ATT	App Tracking Transparency — iOS consent gate for cross-app tracking (IDFA)
Cold start	Launch when the app process must be created from scratch (slowest)
Deferred deep link	Deep link that survives an install: user lands on the intended screen on first open
FCM	Firebase Cloud Messaging — Google's cross-platform push delivery layer
Feature flag	Server-controlled toggle to enable/disable shipped code without a new build
IDFA	Identifier for Advertisers — iOS device ad ID, gated by ATT
Kill switch	Remote toggle to instantly disable a broken feature/endpoint
LTV	Lifetime Value — total revenue expected from a user; compared against CAC
MMP	Mobile Measurement Partner (AppsFlyer/Branch/Adjust) — attribution & deep-link platform
Notification channel	Android category users control independently (since 8.0)
Phased / staged rollout	Releasing an update to a growing % of users to limit blast radius
R8 / ProGuard	Android code shrinking, obfuscation & resource removal
SKAdNetwork	Apple's privacy-preserving, aggregated install-attribution framework
Staged rollout halt	Pausing a rollout to stop more users getting a bad build (not a downgrade)
Stickiness	DAU/MAU ratio — how habitual usage is
Universal Links / App Links	Verified HTTPS links that open the app directly (iOS / Android)
versionCode	Android integer that must strictly increase each Play release

A CLOSING NOTE ON HOW TO USE THIS

This is a living reference. Store policies, commission terms, privacy frameworks, and platform APIs change frequently — treat the concepts here as durable and verify the specifics (percentages, size thresholds, exact API names, current rollout mechanics) against official Apple and Google documentation before you act on them.